



CSE113

Foundations of Computer Science

Course Overview

State University of New York, Korea

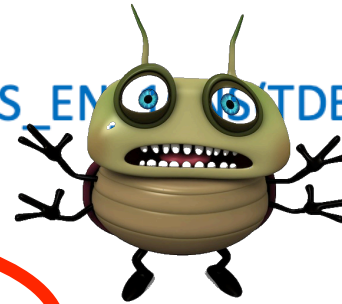
Why Studying Computer Science?

French Ariane 5 Rocket, 1996



Ada code for Ariane 5 Rocket

```
if L_M_BV_32 > 32767 then
  P_M_DERIVE(T_ALG.E_BV) := 16#7FFF#;
elsif L_M_BV_32 < -32768 then
  P_M_DERIVE(T_ALG.E_BV) := 16#8000#;
else
  P_M_DERIVE(T_ALG.E_BV) := UC_16S_EN_16NS(TDB.T_ENTIER_16S(L_M_BV_32));
end if;
P_M_DERIVE(T_ALG.E_BH) :=
  UC_16S_EN_16NS(TDB.T_ENTIER_16S((1.0/C_M_LSB_BH)*G_M_INFO_DERIVE(T_ALG.E_BH)));
```



10-year R&D, \$7,000,000,000 Floating-point Overflow

By comparison, the Korean Nuri project took over a decade and cost ~2 trillion KRW (\$1.5 billion) to develop.

- How to make reliable software?
- How to make efficient software?
- How to make energy-friendly software?
- We need to understand **deeply** how code works

- We need to understand **deeply** how code works
- Quiz: If Precondition holds, will Postcondition holds after executing this piece of code?

```
Precondition:  $x \geq 0$ ;
```

```
z = 0;
```

```
if (x != 0)
```

```
    z = x;
```

```
} else {
```

```
    z = z+1
```

```
}
```

```
Postcondition:  $z > 0$ ;
```

- Yes. We can **prove** it!

Precondition: $x \geq 0$;

$z = 0$;

$\{ x \geq 0 \ \&\& \ z = 0 \}$

if ($x \neq 0$) {

$\{ x > 0 \ \&\& \ z = 0 \}$

$z = x$;

$\{ x > 0 \ \&\& \ z = x \}$

} else {

$\{ x = 0 \ \&\& \ z = 0 \}$

$z = z + 1$

$\{ x = 0 \ \&\& \ z = 1 \}$

}

Postcondition: $z > 0$;

**Propositional
Logic**

**Predicate
Logic**

Proof

CSE215

Sequences

Sets

Functions

Relations

Expected Learning Outcomes

- An ability to check if a mathematical argument is valid
- An ability to verify the correctness of proofs of some existing theorems and prove some new theorems
- An ability to use the mathematical concepts of sequences, functions, relations, and sets in solving computing problems

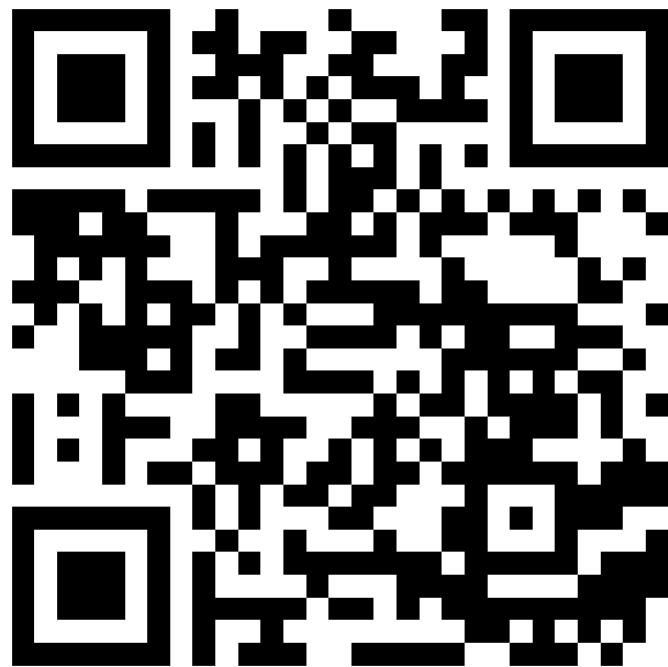
Logistics

Meet the Instructor

- Man email: <zhoulai dot fu at sunykorea dot ac dot kr>
- Affiliated faculty with CS, SBU and ECE, VT
- CSE113 and CSE307
- Research Interest: Programming Language Theory, Software Security, AI systems; floating-point computation
- Previous Work: France, Spain, US, Denmark
- Education: École Polytechnique, France
- Personal: Happily married; no special hobbies

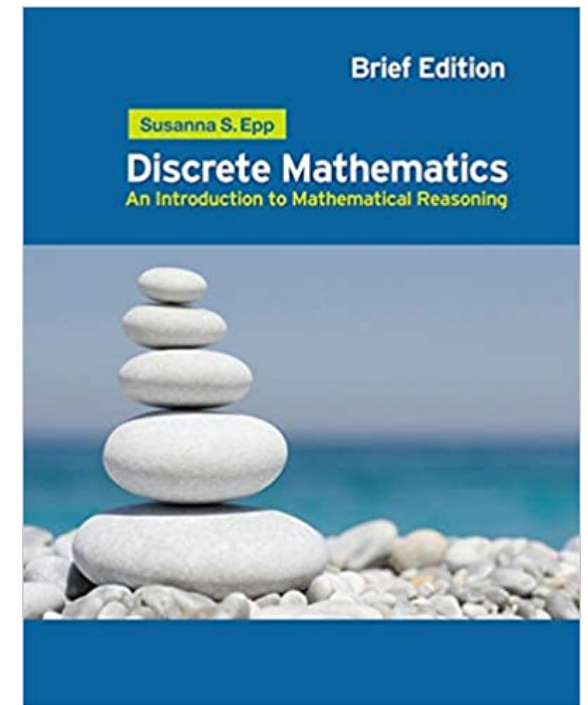
Course website

https://github.com/zhoulai fu/26_cse113_fall

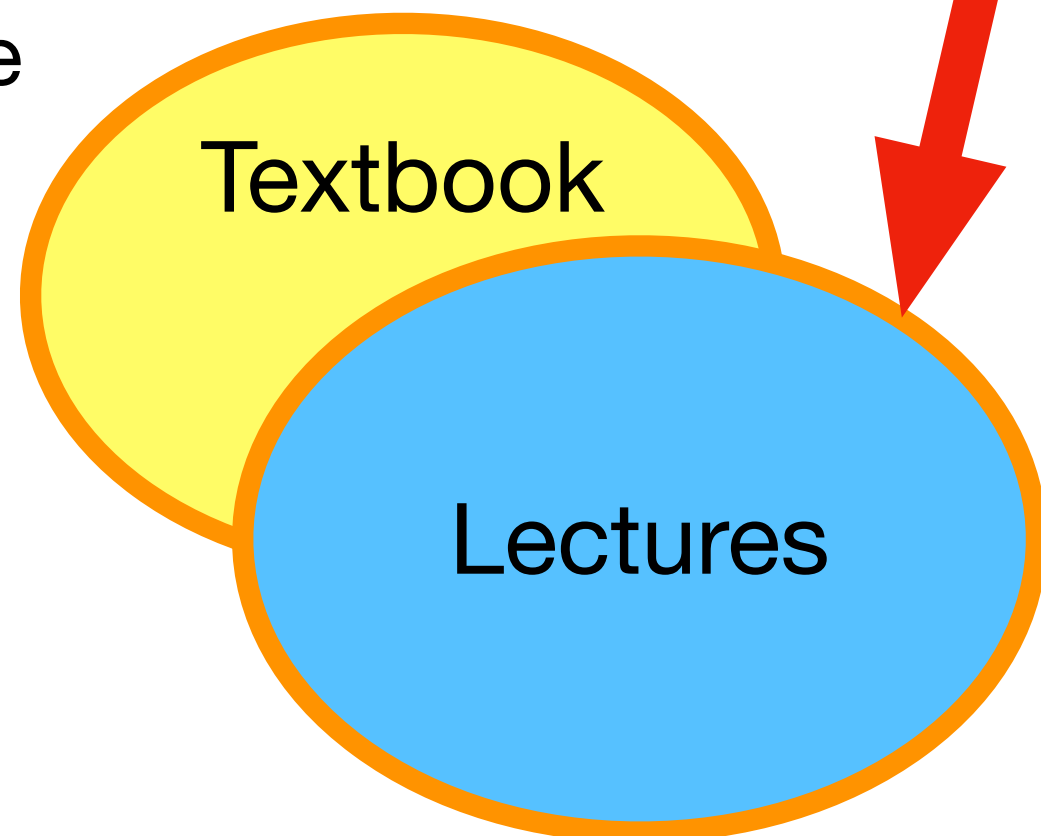


Textbook

- Our course relates to Chapters 2-7
- Very helpful, but optional
- Suggestion: Skim the related chapter before the lecture; read deeper after the lecture
- Textbook may not cover everything in the exams; lectures do



Exam



Questions so far?

Quiz

- Where to find official course info?
- Homework due time?
- How attendance will be checked?
- How late homework will be handled?
- How to email the instructor?
- How grading works?

Course overview

A personal story

- Once upon a time, I worked for a project involving financial calculation
- I needed to sum up a number of floating-point values like
 - $0.1 + 0.2 + 0.3 + 0.7 + 0.9 + 1.2 + 3.5 \dots$
- There were billion of numbers like this, so performance was a key for the project's success
- We decided to use the state-of-the-art multi-core, parallel computing
- Parallel computing works like a divide-and-conquer:
 - $(0.1 + 0.2) + (0.3 + 0.7) + (0.9 + 1.2 + 3.5) + \dots$
- Now, let us think why it looks reasonable to use parallel computing for this task??
- The reason is associative law. $(a + b) + c = a + (b + c)$

A problem

We get different results for each round, if we put parentheses differently each time.

- $(0.1 + 0.2) + (0.3 + 0.7) + (0.9 + 1.2 + 3.5) + \dots$
- becomes different from
- $(0.1 + 0.2 + 0.3) + (0.7 + 0.9) + (1.2 + 3.5) + \dots$

Demo:

<https://www.pythonanywhere.com/try-ipython/>

Why?

- We made this assumption:
 - for any numbers a, b, c , $(a + b) + c = a + (b + c)$
- This is a statement that can be assigned with true or false value, we call it a **proposition**
- The inner part has variables, and can be denoted as a statement with parameters (a, b, c) . We call it a **predicate**.
- Many CS work involves determining if a proposition is true or false. To show the truth is called **to prove**.
- The reason for the problem is that the proposition above is false.

Summary for the story

The whole is called a proposition,
to which we can assign a truth value

$$\forall a, b, c \in I, (a + b) + c = a + (b + c)$$



Predicate

quantifier

variables

set

Summary

- The ultimate goal of this course is to learn fundamentals for understanding why our digital world works or fails.
- We will study logic (propositions, and predicates), proof, and math structures like sets as a language to reason about computer science
- Our next classes will be about propositions.